



Implémentez un Modèle de Scoring
Synthèse

Contexte & Objectifs

PROJET

Construction d'un modèle de credit scoring

- prédiction de la probabilité de défaut de l'emprunteur
- accessibilité du modèle aux utilisateurs finaux
- interprétabilité & reproductibilité des résultats

DÉMARCHE

- model registry avec MLFlow
- model explanations avec Shap
- UI via Streamlit Community Cloud...
- ... sur la base d'une API Flask
- versioning avec GitHub
- unit tests automatisés avec GitHub Actions

➤ Mise en place d'un pipeline CI / CD

Sommaire

1 – ANALYSE EXPLORATOIRE DES DONNÉES

1.1 – NETTOYAGE

1.2 – EXPLORATION

1.3 – FEATURE ENGINEERING

2 – MODÉLISATION

2.1 – MÉTHODOLOGIE

2.2 – SCORE MÉTIER

2.3 – CHOIX DU MODÈLE

3 – MISE EN PRODUCTION

3.1 – DATA DRIFT

3.2 – DÉPLOIEMENT CONTINU

3.3 – API & STREAMLIT APP DÉMOS

CONCLUSION & PERSPECTIVES

ANNEXES

1 – ANALYSE EXPLORATOIRE DES DONNÉES

1.1 – NETTOYAGE

➤ Current applications with Home Credit :

- application_train.csv (305,511 x 122)
- application_test.csv (48,744 x 121)

➤ Previous applications with Home Credit :

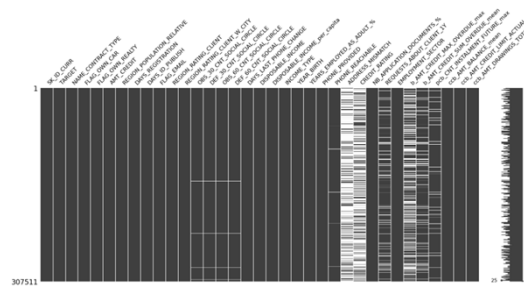
- previous_application.csv (1,670,214 x 37)
- POS_CASH_balance.csv (10,001,358 x 8)
- instalments_payments.csv (13,605,401 x 8)
- credit_card_balance.csv (3,840,312 x 23)

➤ Previous applications with other credit providers :

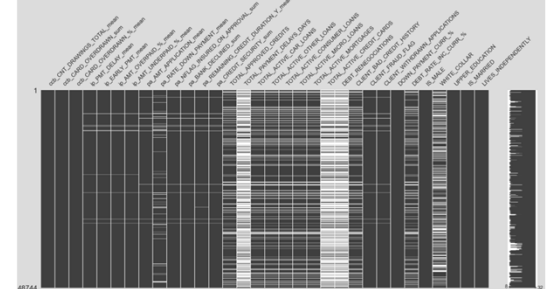
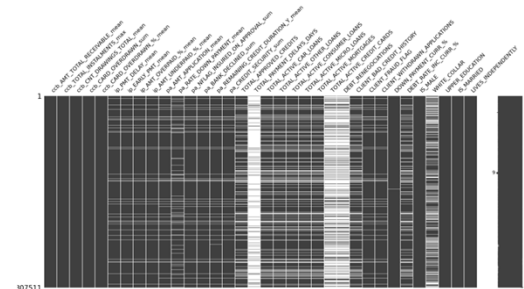
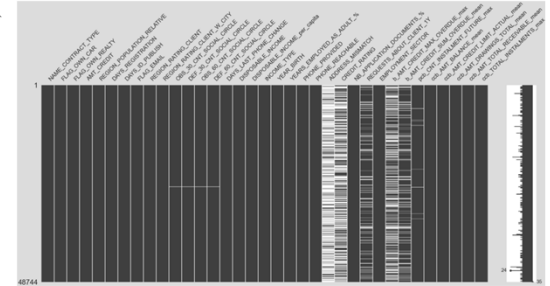
- bureau.csv (1,716,428 x 17)
- bureau_balance.csv (27,299,925 x 3)

Feature engineering, joins & aggregations

➤ Training data (307,511 x 69)

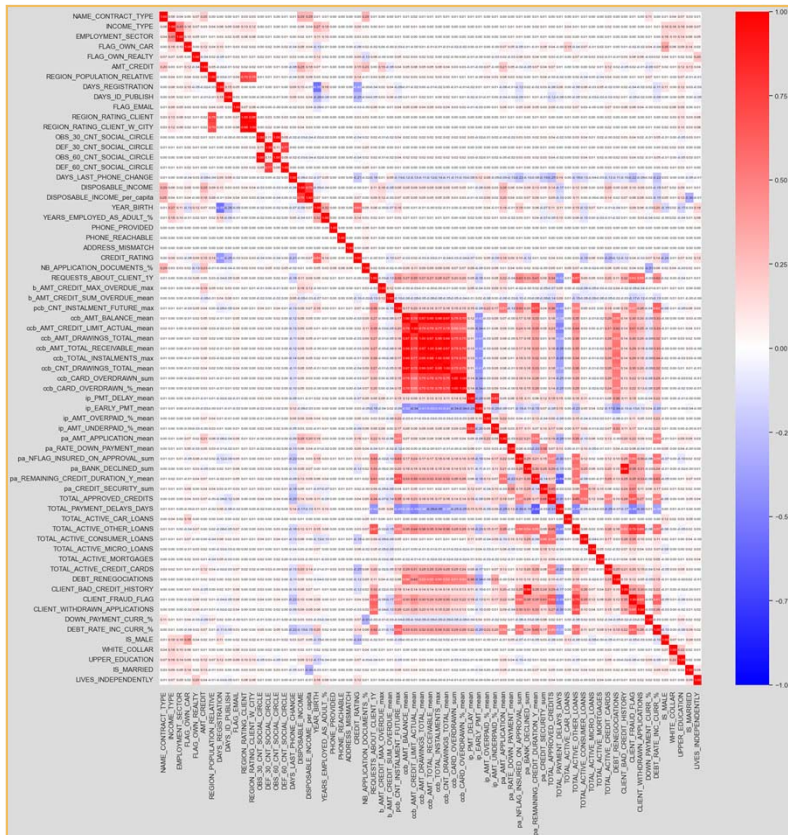


➤ Test data (48,744 x 69)



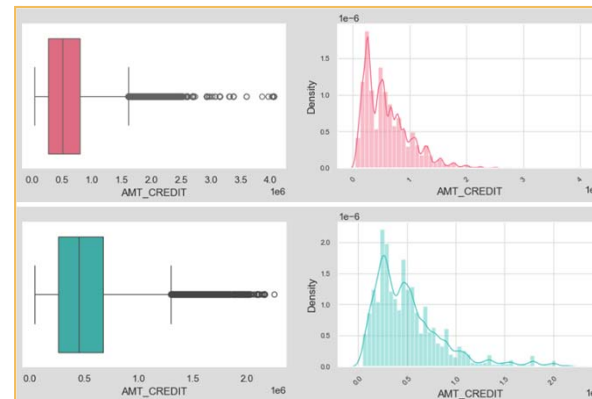
1 – ANALYSE EXPLORATOIRE DES DONNÉES

1.2 – EXPLORATION



Retraitements :
Outliers,
NaNs imputations &
Correlated features

- Training data (307,511 x 56)
- Test data (48,744 x 56)



- Features ne suivent pas une distribution Gaussienne
- Présence de skew...
- ... & de variables excessivement corrélées

1 – ANALYSE EXPLORATOIRE DES DONNÉES

1.3 – FEATURE ENGINEERING

➤ Données socio-économiques

- âge
- genre
- situation maritale
- niveau d'éducation
- secteur d'activité
- revenus par tête du foyer...

56 FEATURES*

- 3 variables catégorielles
- 11 variables booléennes
- 21 variables numériques discrètes
- 22 variables numériques continues

➤ Indicateurs de fraude potentielle

- email/numéro de téléphone joignable
- récence des coordonnées de contact
- suspicion de fraude passée...

➤ Indicateurs de risque de crédit

- credit score
- présence d'une assurance sur les crédits existants
- encours & mensualités totaux des crédits existants
- taux d'endettement
- montant du collatéral
- nombre & montant des défauts de paiement passés...

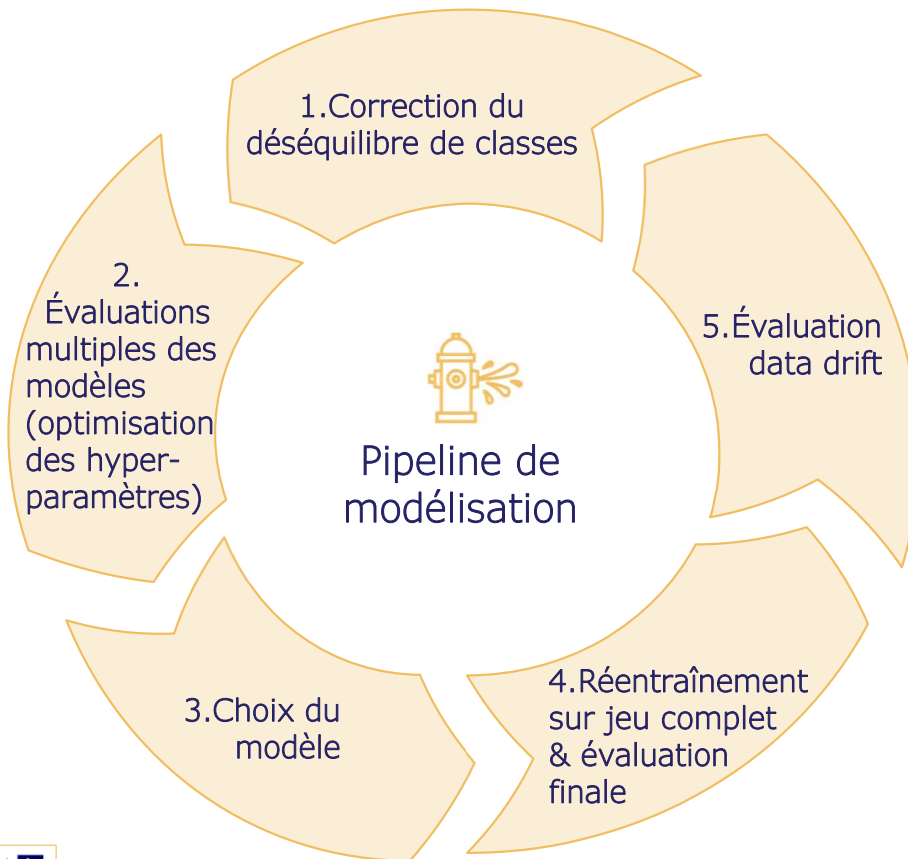
1 TARGET

➤ Défaut de remboursement du crédit

- 8.07% des clients du jeu d'entraînement
- CLASSES DÉSÉQUILIBRÉES

2 – MODÉLISATION

2.1 – MÉTHODOLOGIE



1. TECHNIQUES UTILISÉES:

- Stratified sampling train/val/test
- SMOTE over-sampling
- Random under-sampling

2. MÉTRIQUES D'ÉVALUATION:

- F1 score
- F2 score
- AUC
- Accuracy / balanced accuracy
- Precision / average precision
- Score métier

3. MODÈLES TESTÉS:

- Régression Logistique (baseline)
- LightGBM
- XGBoost

2 – MODÉLISATION

2.2 – SCORE MÉTIER

- Objectif : maximiser le rendement en pénalisant les erreurs de prédiction (faux négatifs et faux positifs) en fonction de leur coût réel et de leur coût d'opportunité respectifs pour la banque :
- Vrais positifs = 0 (prêt refusé => aucun revenu & aucun risque de perte)
 - Vrais négatifs = 0,05 (prêt accordé => gain = marge moyenne 5%)
 - Faux positifs = -0.05 (prêt refusé à tort => coût d'opportunité = marge moyenne 5% perdue)
 - Faux négatifs = -0.5* (prêt accordé à tort => coût réel 10 fois supérieur à celui des faux positifs)
- Fonction de gain : à maximiser

$$\text{Score Métier} = 0 * TP + 0.05 * TN - 0.05 * FP - 0.5 * FN$$



* Coût total estimé suite au défaut du client et après prise en compte des intérêts non-perçus sur la durée résiduelle du prêt au moment du défaut et de la possibilité de recouvrement partiel par saisie et cession des actifs garantis & autres biens mobiliers et/ou immobiliers du client.

2 – MODÉLISATION

2.3 – CHOIX DU MODÈLE

➤ Évaluation sur le jeu de validation :

Model	Validation Metrics										
	F1 Score	F2 Score	AUC	Accuracy	Balanced Accuracy	Precision	Average Precision	Recall	Best k	Score Metier	Optimal Threshold
Regression Logistique	0.2486	0.3969	0.6696	0.6784	0.6696	0.1532	0.1285	0.6590	11	0.6551	0.4690
LightGBM	0.2627	0.4102	0.6814	0.7028	0.6814	0.1642	0.1355	0.6557	13	0.6809	0.4770
XGBoost	0.2311	0.3887	0.6607	0.6169	0.6607	0.1379	0.1215	0.7129	3	0.6696	0.5780

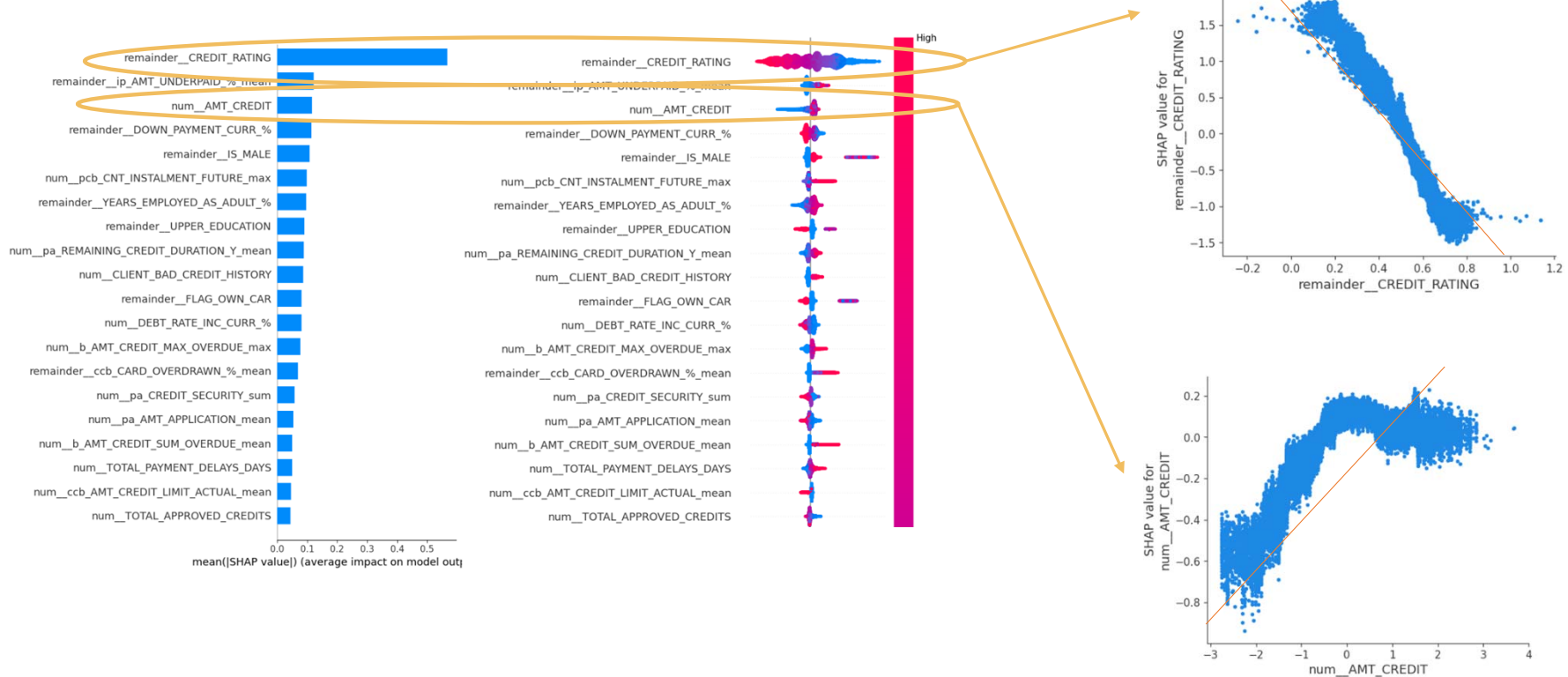
➤ Réévaluation sur le jeu de test :

	Test Metrics										
	F1 Score	F2 Score	AUC	Accuracy	Balanced Accuracy	Precision	Average Precision	Recall	Best k	Score Metier	Optimal Threshold
LightGBM	0.2632	0.4109	0.6819	0.7034	0.6819	0.1646	0.1358	0.6563	13	0.6927	0.502

2 – MODÉLISATION

2.3 – CHOIX DU MODÈLE

➤ Feature importance globale*:

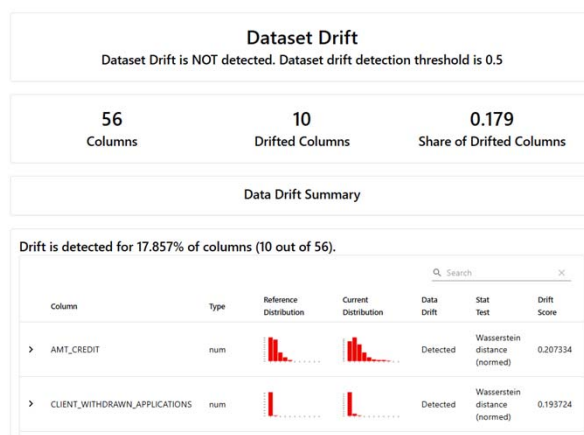


* Pour une analyse de l'importance locale des features, se référer aux prédictions de l'application Streamlit sur les dossiers clients individuels (voir exemple au point 3.3 slide 15).

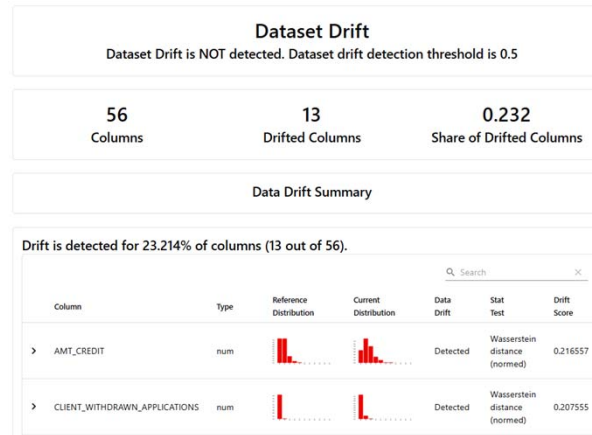
3 – MISE EN PRODUCTION

3.1 – DATA DRIFT

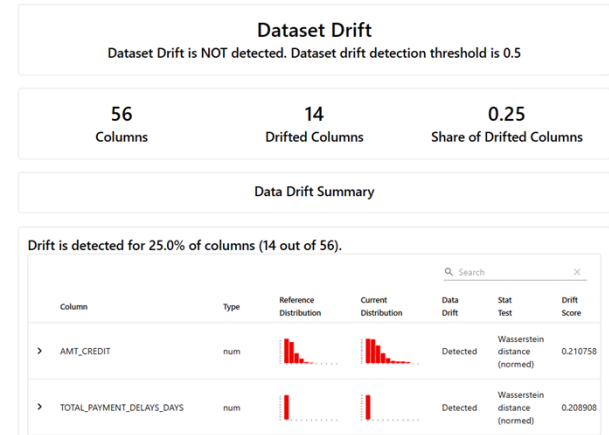
➤ Whole dataset:



➤ Positive Class:

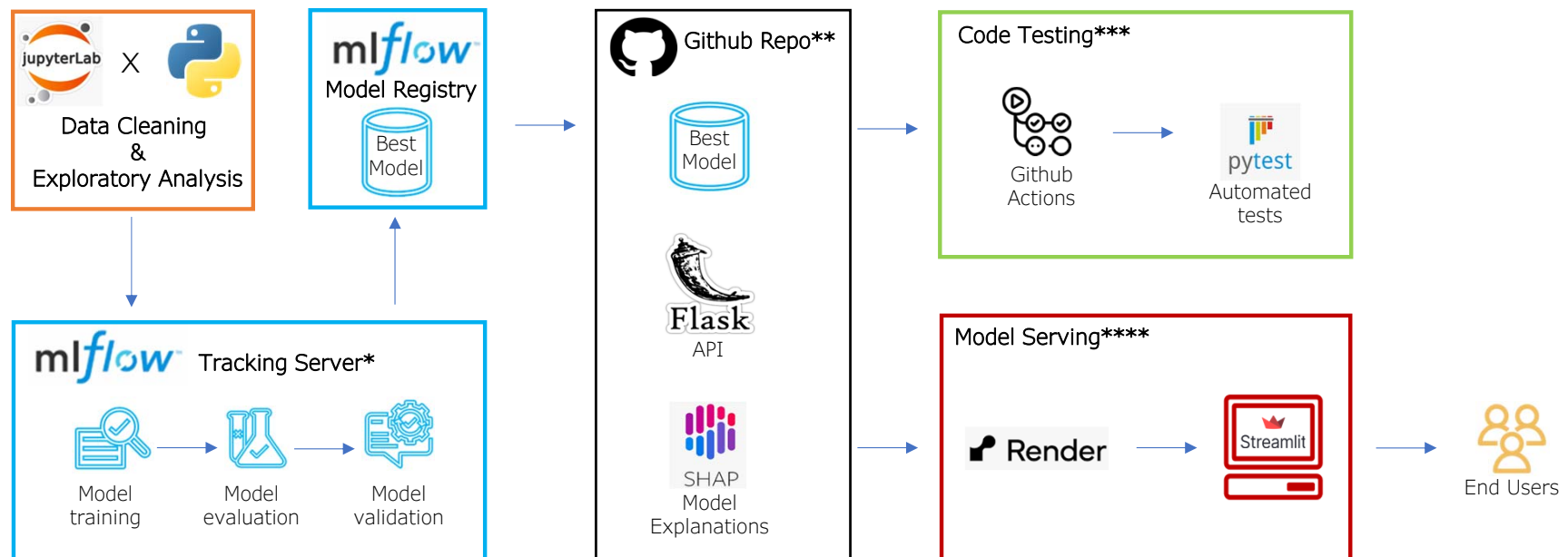


➤ Negative class:



3 – MISE EN PRODUCTION

3.2 – DÉPLOIEMENT CONTINU (1/2)



3 – MISE EN PRODUCTION

3.2 – DÉPLOIEMENT CONTINU (2/2)

➤ GitHub repos :

- Analyse exploratoire & modélisation
<https://github.com/CelineBoutinon/credit-scoring.git>
- API, unit tests & application Streamlit
<https://github.com/CelineBoutinon/credit-scoring-api.git>

➤ API disponible sur :

<https://credit-scoring-api-0p1u.onrender.com>

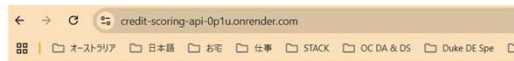
➤ Streamlit App disponible sur :

<https://credit-scoring-api-f6g7yxlaqnt3qhu7rk7rkf.streamlit.app/>

3 – MISE EN PRODUCTION

3.3 – API – CLOUD DÉMO

➤ API landing page :



Credit Scoring

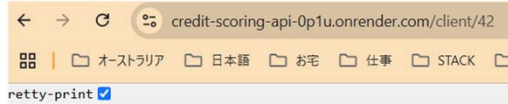
Welcome to the HOME CREDIT Credit Scoring app.

- use /client/ID to access client demographics

- use /predict/ID to retrieve client credit application decision

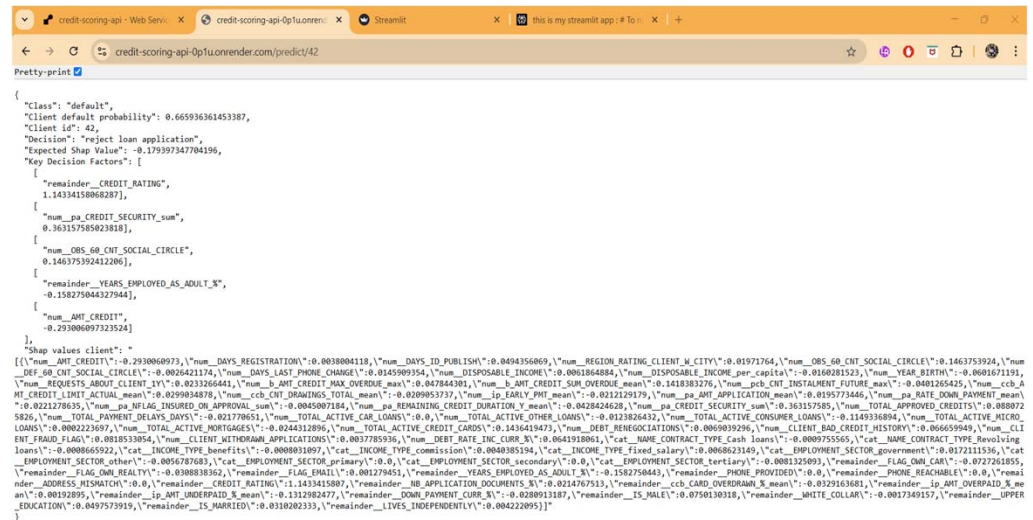
where ID is the client's unique Home Credit application number (whole number between 1 and 46128)

➤ API client demographics example :



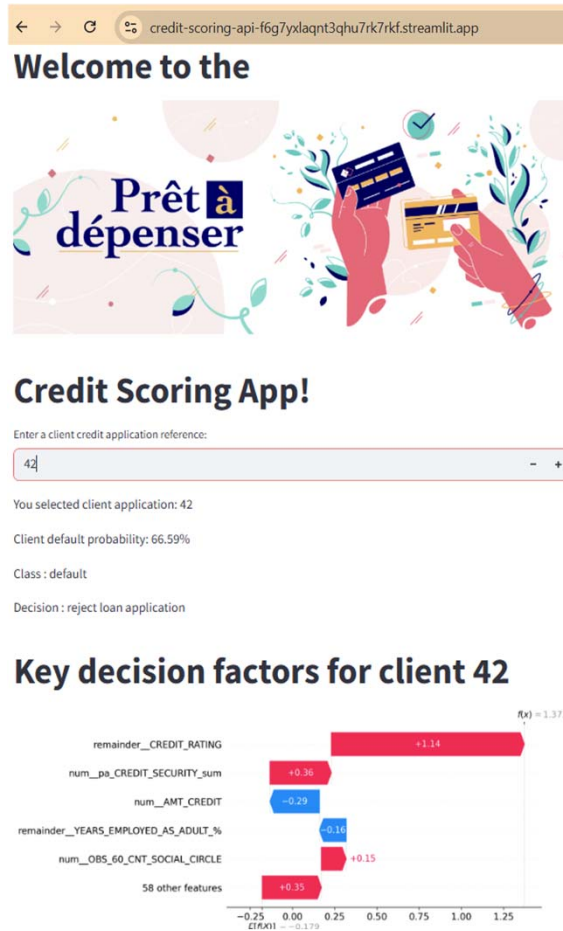
```
"Client Home Credit application number: ",
42, "Client summary information:",
{
  "AGE": 23,
  "CLIENT_BAD_CREDIT_HISTORY": "no",
  "CLIENT_FRAUD_FLAG": "yes",
  "CREDIT_RATING": 0.24272203195195,
  "DISPOSABLE_INCOME_per_capita": 121158,
  "EMPLOYMENT_SECTOR": "tertiary",
  "INCOME_TYPE": "fixed_salary",
  "IS_MALE": "yes",
  "IS_MARRIED": "no",
  "LIVES_INDEPENDENTLY": "yes",
  "UPPER_EDUCATION": "no",
  "WHITE_COLLAR": "no"
}
```

➤ API prediction example :



3 – MISE EN PRODUCTION

3.3 – STREAMLIT APP – CLOUD DÉMO



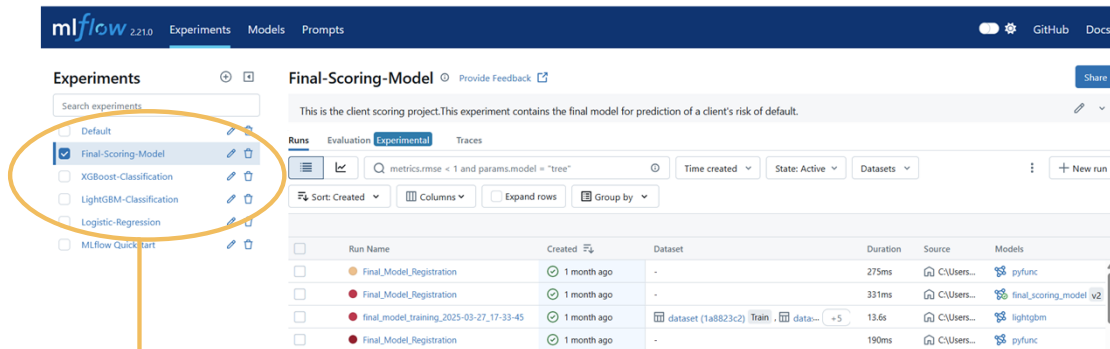
Conclusion & perspectives

- Déséquilibre de classes nécessite des ajustements pour une modélisation pertinente...
- ... et pour le suivi des modèles en production:
 - détection anticipée du data drift
 - nécessité de monitorer aussi le target drift
- Coût des erreurs de prédiction étroitement lié au contexte métier
 - ex. épidémiologie/virologie vs. finance – attention à ne pas minimiser les faux positifs
 - coûts individuels & collectifs réels, pas seulement des coûts d'opportunité

ANNEXES

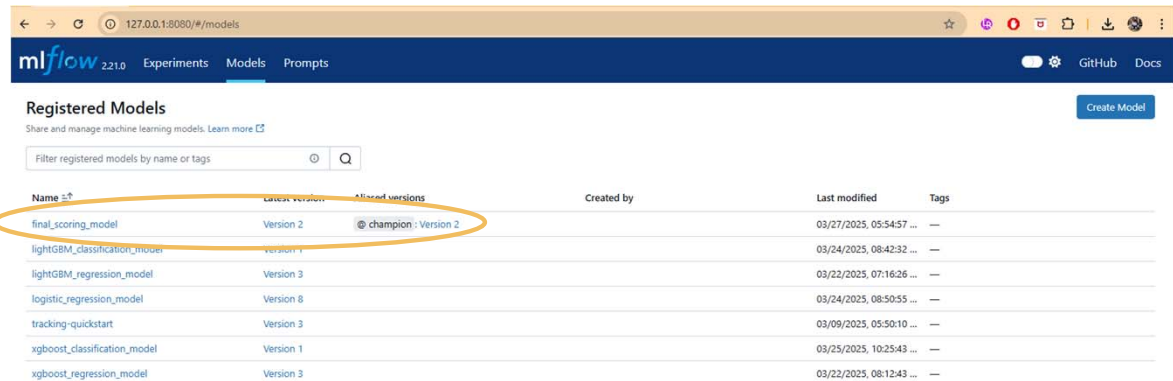
Annexe 1 - MLFlow Tracking Server

➤ Experiment Tracking :



The screenshot shows the MLFlow Tracking Server interface. The 'Experiments' sidebar on the left lists several experiments, with 'Final-Scoring-Model' selected and circled in orange. The main panel displays the details for the 'Final-Scoring-Model' experiment, including a description, a search bar, and a table of runs. The table has columns for Run Name, Created, Dataset, Duration, Source, and Models. The runs listed are 'Final_Model_Registraton', 'Final_Model_Registraton', 'final_model_training_2025-03-27_17-33-45', and 'Final_Model_Registraton'.

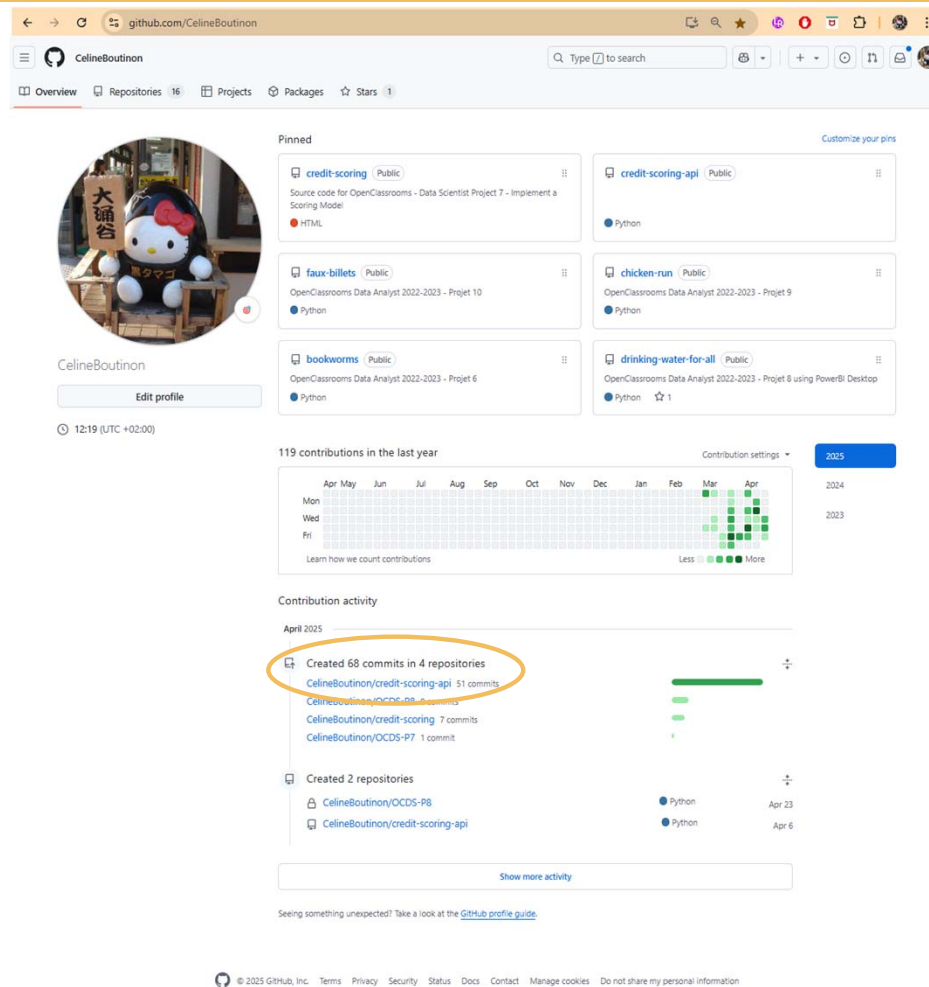
➤ Model Registry Logging :



The screenshot shows the MLFlow Tracking Server interface with the 'Models' tab selected. The 'Registered Models' table lists several models, with 'final_scoring_model' circled in orange. The table has columns for Name, Latest Version, Alloted versions, Created by, Last modified, and Tags. The models listed are 'final_scoring_model', 'lightGBM_classification_model', 'lightGBM_regression_model', 'logistic_regression_model', 'tracking-quickstart', 'xgboost_classification_model', and 'xgboost_regression_model'.

Name	Latest Version	Alloted versions	Created by	Last modified	Tags
final_scoring_model	Version 2	@ champion : Version 2		03/27/2025, 05:54:57 ...	—
lightGBM_classification_model	Version 1			03/24/2025, 08:42:32 ...	—
lightGBM_regression_model	Version 3			03/22/2025, 07:16:26 ...	—
logistic_regression_model	Version 8			03/24/2025, 08:50:55 ...	—
tracking-quickstart	Version 3			03/09/2025, 05:50:10 ...	—
xgboost_classification_model	Version 1			03/25/2025, 10:25:43 ...	—
xgboost_regression_model	Version 3			03/22/2025, 08:12:43 ...	—

Annexe 2 - Commit history as of 25/04/25



Annexe 3 - Unit Tests implementation with GitHub Actions

The test_api.py file is triggered by a workflow upon each push to the repo according to the config in pytest.ini

The screenshot shows the GitHub repository 'credit-scoring-api' with a commit history table. The table lists files and their commit messages, with the 'tidy code' commit being the most recent. A workflow run 'build' is shown as successful, with a 'Details' link. A callout box points to the 'tidy code' commit and the 'test_api.py' file. Another callout box points to the 'build' workflow run. A third callout box points to the 'test_api.py' file in the commit history table.

File	Commit Message	Commit Time
tidy code	tidy code	27 minutes ago
test_api.py	tidy code	27 minutes ago
streamlit_cloud_app_vf.py	tidy code	27 minutes ago
streamlit_cloud_app_v6.py	add pytests for streamlit app	2 days ago
shap_values_testjoblib	further improve app	last week
shap_values_test.csv	test update nfrom VS Code extension	last week
requirements.txt	update requirements.txt	19 hours ago
pytest.ini	tidy code	27 minutes ago
optimal_threshold.joblib	additional app improvements	last week
logo.png	add streamlit app	last week
final_model.joblib	update app and load model	last week
expected_value.joblib	update app and load model	last week
X_test_final.csv	update app and load model	last week
.gitignore	add streamlit app	last week
.gitattributes	add git attributes	last week
__pycache__	rename api to app	last week
.github/workflows	Create test-app.yml	yesterday

build
succeeded 8 minutes ago in 40s

- Set up Job
- Run actions/checkout@v4
- Set up Python 3.10
- Install dependencies
- Lint with RakeB
- Test with pytest
- Post Set up Python 3.10
- Post Run actions/checkout@v4
- Complete job

```
1 Run pytest
2 ----- test session starts -----
3 platform linux -- Python 3.10.10, pytest-8.3.5, pluggy-1.5.0
4 rootdir: /home/runner/work/credit-scoring-api/credit-scoring-api
5 configfile: pytest.ini
6 plugins: anyio-4.0.0
7 collected 7 items
8
9 test_api.py ..... [100%]
10
11 ----- 7 passed in 2.34s -----
12
13 Post Set up Python 3.10
14
15 Post Run actions/checkout@v4
16
17 Complete job
18 Cleaning up orphan processes
```

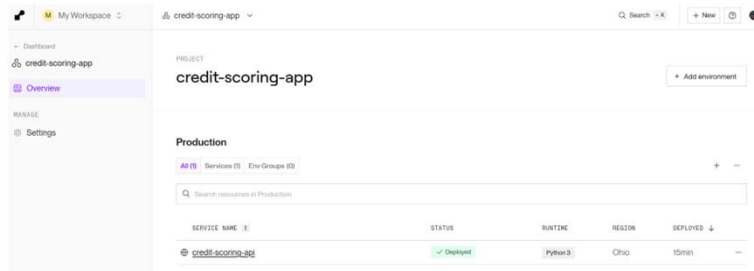
7 tests successfully passed in accordance with local results :

```
PS C:\Users\celin> cd "D:\Projects\Python\OCDS-repos-all\credit-scoring-api"
PS C:\Users\celin\DS Projects\Python\OCDS-repos-all\credit-scoring-api> py -m pytest test_api.py -v -s
platform win32 -- Python 3.13.2, pytest-8.3.5, pluggy-1.5.0 -- C:\Users\celin\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\celin\DS Projects\Python\OCDS-repos-all\credit-scoring-api
configfile: pytest.ini
plugins: anyio-5.0.0, Faker-37.1.0
collected 7 items

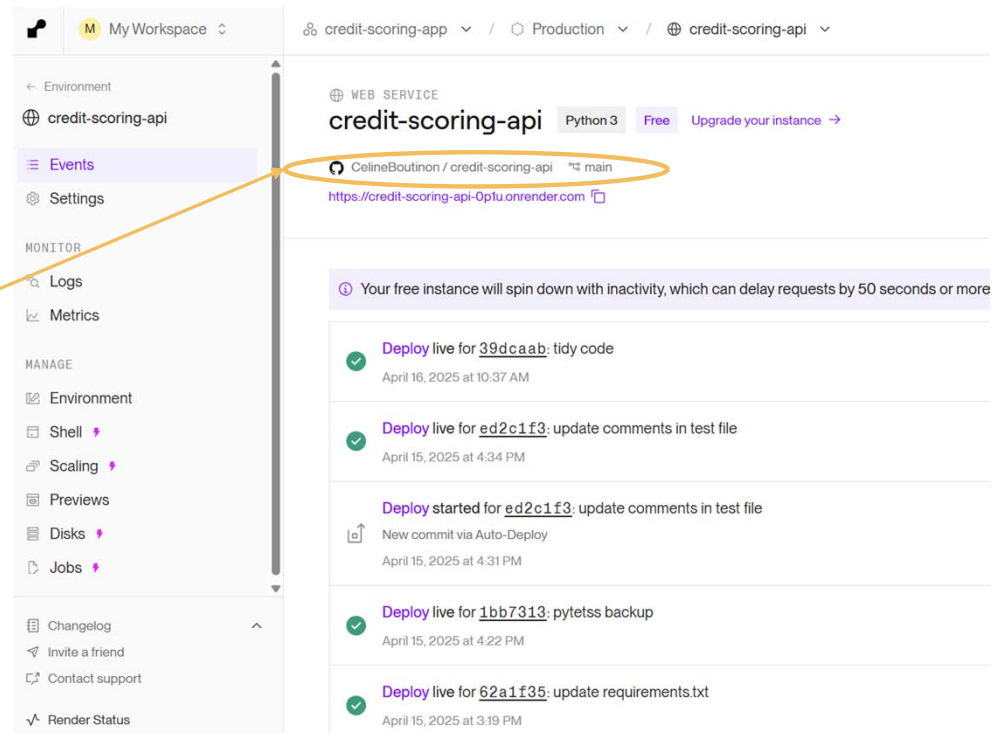
test_api.py::test_prediction PASSED
test_api.py::test_client_demographics PASSED
test_api.py::test_client_demographics_invalid_id PASSED
test_api.py::test_prediction_invalid_id PASSED
test_api.py::test_prediction_json_structure PASSED
test_api.py::test_model_loading PASSED
test_api.py::test_csv_loading PASSED

===== 7 passed in 6.51s =====
```

Annexe 4 – API Cloud deployment with Render.com



The API auto-deploys to Render.com upon each push to GitHub Repo credit-scoring-api



Annexe 5 – App Cloud deployment with Streamlit Community Cloud

The screenshot shows the Streamlit Community Cloud interface. At the top, the user 'celineboutinon' is logged in. Below the navigation bar, the 'My apps' tab is selected, showing a list of apps. The app 'credit-scoring-api' is highlighted. A modal window titled 'App settings' is open, showing the 'General' tab. The 'App URL' field is circled in orange, displaying 'credit-scoring-api-f6g7yqlaqt3qhu7rk7rkf.streamlit.app'. The 'Python version' is set to '3.10'. A 'Save changes' button is at the bottom right. A text box on the left explains that the app auto-refreshes upon each push to the GitHub Repo 'credit-scoring-api'.

celineboutinon's apps

credit-scoring-api · main · streamlit_cloud_app_vf.py

App settings

- General
- Sharing
- Secrets

App URL
Pick a custom subdomain for your app's URL. The default URL is based on the app's location in GitHub.

credit-scoring-api-f6g7yqlaqt3qhu7rk7rkf .streamlit.app

Python version
Choose which Python version to run your app.

3.10

Save changes

The app auto-refreshes upon each push to GitHub Repo credit-scoring-api